Google Cloud H2S

AI Builder Cup

Build in JAPAC.
Build for the world.

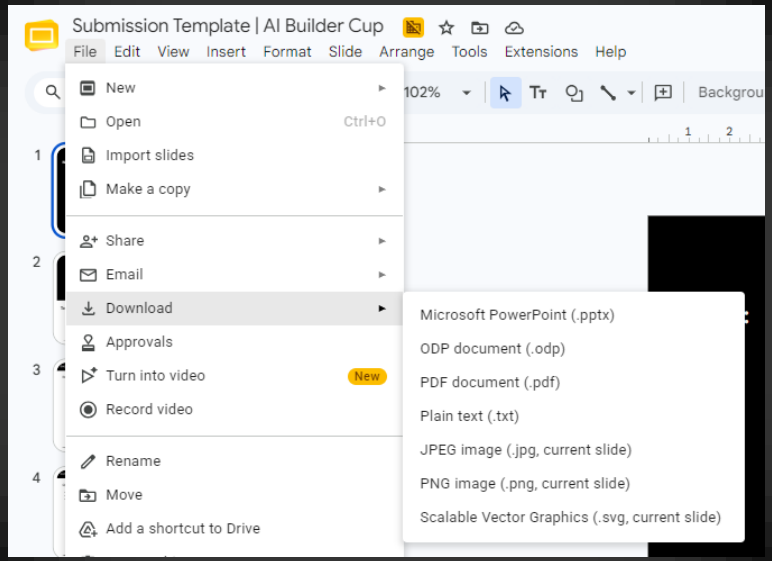


Team Details

Team name: Kisan Mitra

Team leader name: Kavita Chegaraddi

Problem Statement: Sustainability & Social Impact — Smallholder Crop Advisory Agent for tomato farmers in the Kolar belt, Karnataka



Note: Please download this template to make your edits and submit the final version.

To download, go to File > Download > Microsoft PowerPoint (.PPTX). Kindly refrain from sending edit requests.

Brief about the idea

- A smallholder tomato farmer in the Kolar belt, Karnataka spots a diseased leaf with no agronomist to call and unreliable connectivity to search with.
- Existing diagnosis apps (Plantix, PlantVillage) identify the disease but stop there — they don't say where to buy the treatment nearby, what it costs, or whether a nearby outbreak means the farmer should check sooner.
- Kisan Mitra turns a photo and a spoken description into a cited diagnosis, a spoken Kannada advisory, the nearest in-stock dealer with today's price, a proactive outbreak alert, a crop-loss insurance evidence packet, a group-buy discount alert, and an anonymized district dashboard for extension officers.
- Target users: smallholder tomato farmers in the Kolar tomato belt, Karnataka — typically 0.5–4 acre plots, variable literacy, intermittent connectivity, no dedicated access to an agronomist. Primary language: Kannada.

Opportunities

- How different is it: Plantix diagnoses, AgroStar connects to dealers, PlantVillage pairs AI with weather — none combine diagnosis, voice, and market linkage in one flow, and none reason across farmers or across time on the same plot.
- How it solves the problem: the outbreak alert reaches a farmer before their own crop shows damage; the insurance claim assistant turns a diagnosis into a fileable claim; the group-buy alert turns an early warning into a coordinated bulk discount; the district dashboard turns farmer-level data into a resourcing signal for an extension officer or NGO.
- USP: moves the system from advice to economic action and policy support — no existing tool in this space does all four.

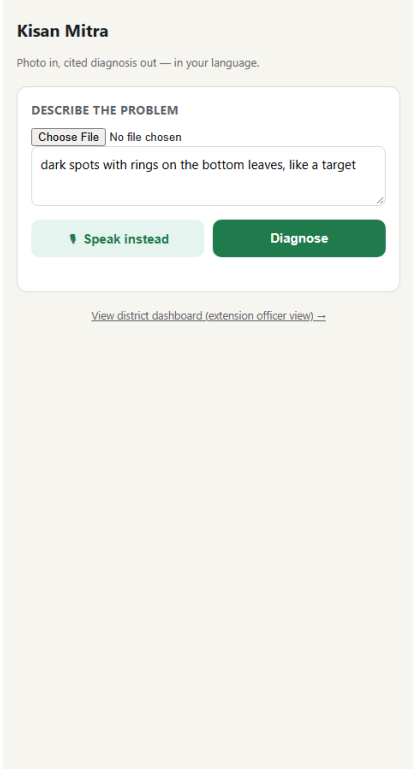
List of features offered by the solution

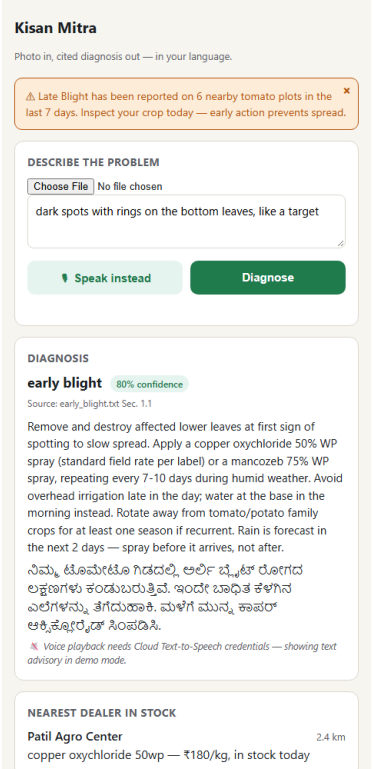
- Cited disease diagnosis from a photo + spoken description — not a black-box guess
- Spoken advisory in Kannada, weather-aware (holds off spraying before forecast rain)
- Nearest in-stock dealer lookup with today's price
- Proactive outbreak early-warning — alerts nearby farmers before their own crop shows symptoms
- Plot-level memory — flags recurring disease as a root-cause issue (e.g. drainage), not just a repeat treatment
- Sell-side market linkage — best nearby mandi price when a healthy crop is near harvest
- Crop-loss insurance claim assistant — auto-generates a fileable evidence packet
- Group-buy / collective bargaining alert — coordinates nearby farmers for a bulk discount
- District dashboard — anonymized, village-level view for an extension officer or NGO
- Gemini-driven orchestration — the model itself decides which of these tools to call, via real function-calling, not a hardcoded decision tree

Process flow diagram or Use-case diagram

- Farmer speaks/types symptom + optionally uploads a photo, entirely in the browser
- → Gemini multimodal diagnosis call, grounded by RAG retrieval (TF-IDF, hand-rolled in JS) over bilingual English + Kannada disease guides shipped with the site
- → Weather tool call (Open-Meteo) checked before recommending a spray
- → A second Gemini call, using real function-calling, decides WHICH of 4 tools to invoke (dealer lookup, sell-price lookup, insurance-claim check, group-buy check) based on the diagnosis, outbreak signal, and near-harvest flag — not a hardcoded if/else
- → Chosen tools run against the known dealer/pricing data; results assembled
- → Advisory translated to Kannada, spoken via the browser's own Web Speech API if a kn-IN voice is available
- → Response rendered: diagnosis, dealer card, outbreak banner, insurance packet, group-buy card
- → Separately: district dashboard loads a pre-aggregated, anonymized village-level snapshot built at deploy time, never raw farmer data

Wireframes / Mock diagrams of the proposed solution





Architecture diagram of the proposed solution

- 100% client-side static site — no backend server, no billing account, no credit card anywhere in the deployed stack — Firebase Hosting (free Spark plan) serves the app
- Diagnosis: a real Gemini multimodal API call direct from the browser, grounded by a hand-rolled TF-IDF + cosine-similarity RAG index over 18 disease documents (9 English + 9 Kannada), so a Kannada symptom description retrieves relevant passages natively
- Orchestration: a second Gemini call uses real function-calling (tools/functionDeclarations) to decide which of 4 downstream tools to invoke — the model drives the decision, not a hardcoded branch — falling back to equivalent deterministic rules only if the live call fails after retries
- Weather: a direct, keyless call to the free Open-Meteo API (confirmed CORS-enabled)
- Dealer directory, mandi prices, and diagnosis history ship as static JSON/CSV with the site; the district dashboard loads a pre-aggregated snapshot built at deploy time from the private farmer dataset, so raw farmer/plot linkage never ships publicly
- Voice advisory: the browser's own Web Speech API (SpeechSynthesis), feature-detecting a kn-IN voice at runtime

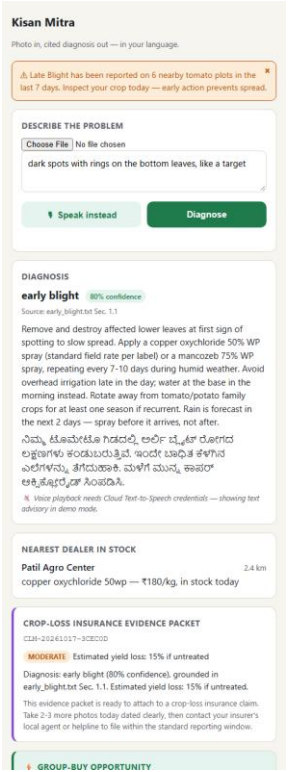
Technologies to be used in the solution

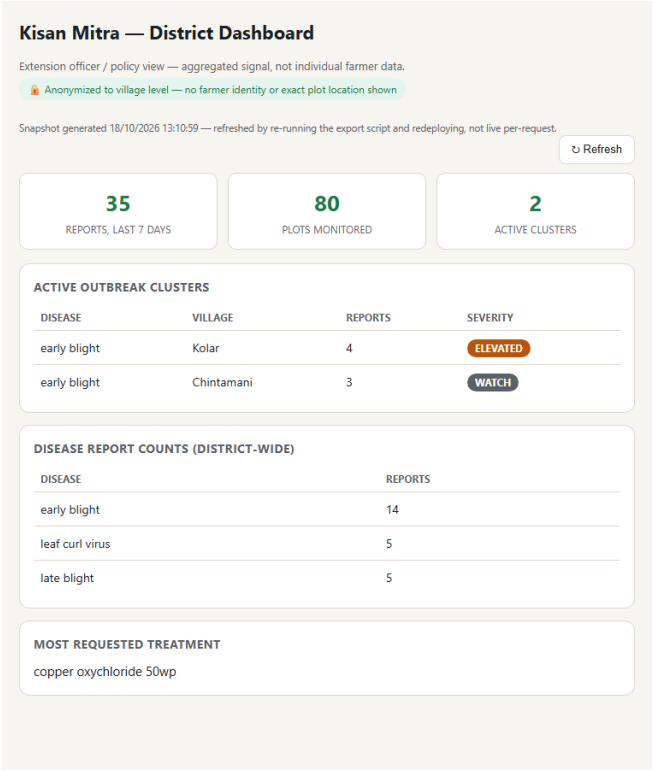
- Gemini API (gemini-3.8-flash) — called directly from the browser for two things: multimodal diagnosis, and a second call using real function-calling to decide which tools to invoke
- Firebase Hosting — free Spark plan, zero billing, serves the entire static site
- Vanilla JavaScript (ES modules) — no framework, no bundler, no agentic framework (no LangChain/CrewAI/Vertex Agent Builder) — direct fetch() calls to the Gemini API
- Hand-rolled TF-IDF + cosine similarity (JS) — RAG retrieval over the disease guides, no vector database or external search service
- Open-Meteo — free, keyless weather API
- Web Speech API (browser-native) — speech recognition for Kannada voice input, SpeechSynthesis for voice output
- Static JSON/CSV data files — dealer directory, mandi prices, diagnosis history
- Python (data generation, build-time dashboard aggregation), Playwright (verification)

Estimated implementation cost

- Current deployed cost: $0 — no billing account, no credit card, anywhere in the stack
- Firebase Hosting: free Spark plan, no usage cap reached at this scale
- Gemini API: free tier via Google AI Studio — 20 requests/day per project for gemini-3.8-flash; each diagnosis now uses 2 calls (diagnosis + orchestration), so roughly 10 full diagnoses/day before the app gracefully falls back to deterministic logic for the rest of the day
- Open-Meteo, Web Speech API: free, no quota observed in testing
- Scaling beyond the free tier: upgrading to a paid Gemini tier is the only cost that would be needed to raise the daily diagnosis cap — no other paid service is required even at higher usage, since there is no backend, database, or search service to scale

Snapshots of the prototype





Prototype Performance report / Benchmarking

- Measured live in the browser against the deployed static site (5 calls per operation):
- RAG retrieval (TF-IDF index build, cold) — ~92ms once per session; warm lookups <1ms
- Dealer / sell-price lookup — ~0.1–2.5ms (in-memory JSON lookups)
- Outbreak check (geo-radius over 300 diagnosis records) — ~3.6ms avg
- District dashboard snapshot load — ~17ms avg
- Weather (live Open-Meteo call) — ~660ms avg (external network call)
- Full diagnosis pipeline (2 Gemini calls: diagnosis + orchestration) — several seconds under normal quota; up to ~20s worst case if quota is exhausted and both calls exhaust retries before falling back
- Verified end-to-end with Playwright against the live deployed URL, with zero local servers running — confirms zero hidden dependency on anything other than the public Gemini and Open-Meteo APIs

Additional Details / Future Development

- Next step — upgrade to managed Google Cloud services once billing is available: Gemini moved behind a Cloud Run/Cloud Functions proxy (closes the current public-key exposure and removes the 20-req/day free-tier ceiling), Firestore replacing the static dealer/price/history JSON so the district dashboard becomes genuinely live instead of a deploy-time snapshot, Vertex AI Search replacing the hand-rolled TF-IDF retrieval, and Cloud Translation/Text-to-Speech replacing the template-based Kannada advisory. Every substitute today sits behind one isolated call site per module, so each swap is incremental, not a rewrite.
- Offline-first fallback (planned): a distilled on-device TensorFlow Lite classifier for the 8–10 most common regional diseases, so core diagnosis keeps working when connectivity drops entirely. Training script already written, pending a real multi-hour training run on the full PlantVillage dataset.
- Expansion path: adding a new crop or region means indexing a new disease guide into the retrieval layer, not rewriting the diagnosis logic — the architecture is crop-agnostic by design even though this build is scoped to tomato in Karnataka for the demo.
- Resource & environmental impact: targeted treatment instead of blanket spraying, weather-timed application to reduce chemical runoff, and earlier detection to reduce crop loss — moving this beyond social impact into genuine sustainability.

Provide links to your:

GitHub Public Repository: https://github.com/Kavitacs08/kisan-mitra

Demo Video Link (3 Minutes): [add link before submission]

Final Product Link: https://kisan-mitra-2026-56c5d.web.app

Google Cloud H2S

AI Builder Cup

Build in JAPAC.
Build for the world.

Thank you

